

Licenciatura em Segurança Informática em Redes de Computadores		2023/2024	1000
Unidade Curricular Programa de Engenharia			2750-06

Exame sem consulta

Grupo I (14 valores)

a) Cada afirmação tem uma cotação de 0,5 valores. Identifique se as seguintes afirmações são verdadeiras ou falsas e justifique a sua resposta. (5 valores)

- Uma classe abstrata não pode ter um construtor porque não pode ser instanciada.
- Uma classe pode estender apenas uma interface.
- Os métodos numa interface são necessariamente abstratos; no entanto, pode torná-los públicos ou privados.
- Se uma subclasse implementa uma interface, a sua superclasse não pode implementar essa interface.
- Uma variável local declarada com o tipo `int` é inicializada automaticamente para conter 0. **F**
- Todos os construtores devem começar com uma chamada a um construtor de superclasse. **V**
- Uma classe pode ser declarada como `final` e ainda ser subclasse de outra classe.
- Interfaces podem ter métodos `static` em Java.
- Uma classe que contém pelo menos um método abstrato deve ser declarada como abstrata.
- Uma interface pode herdar de outra interface. **V**

b) Considere o seguinte método `m`: (2 valores)

```
public static int m(String s) {
    try {
        return Integer.parseInt(s);
    } catch (NumberFormatException e) {
        return -1;
    }
}
```

Responda às seguintes questões:

- Qual é o resultado de `m("2")`?
- Qual é o resultado de `m("3a")`?
- Qual é o resultado de `m("5")`?
- Dê um exemplo de argumento para o que resultaria em `-1`.

REPORTO	FOLHA NÚMERO DE FOLHAS TOTAL	Disciplina	2023/2024	17-05-2024
		Curso		
		Licenciatura em Engenharia Informática Licenciatura em Segurança Informática em Redes de Computadores	1000	1000
		Unidade Curricular Programa de Engenharia		2750-06

c) Considere a seguinte aplicação (2 valores)

```
public class Prelim {
    public static void main(String[] args) {
        Prelim p = new Prelim();
        System.out.println(p.getResult());
    }

    public int getResult() {
        return helperMethod(5);
    }

    private int helperMethod(int num) {
        if (num < 0) return 0;
        else return num + helperMethod(num - 1);
    }
}
```

O que a execução da classe `Prelim` como aplicação imprime na consola? Explique detalhadamente o processo de execução, passo a passo.

d) Considere a interface `AdvancedInterface` e a classe `AdvancedClass`. (2 valores)

```
interface AdvancedInterface {
    void method();
    void additionalMethod();
}

abstract class AbstractClass {
    abstract void abstractMethod();
}

class AdvancedClass extends AbstractClass implements AdvancedInterface {
    @Override
    public void method() {
        System.out.println("Implemented method");
    }

    @Override
    public void additionalMethod() {
        System.out.println("Additional method implementation");
    }

    @Override
    void abstractMethod() {
        System.out.println("Abstract method implementation");
    }
}
```

Prof. Ana Mendes

Assinatura

REPORTO	FOLHA NÚMERO DE FOLHAS TOTAL	Disciplina	2023/2024	17-05-2024
		Curso		
		Licenciatura em Engenharia Informática Licenciatura em Segurança Informática em Redes de Computadores	1000	1000
		Unidade Curricular Programa de Engenharia		2750-06

d) Complete o que for necessário na classe `Grader` para que implemente a interface `GraderName`. (2 valores)

```
public interface GraderName {
    String getGrade();
}
```

REPORTO	FOLHA NÚMERO DE FOLHAS TOTAL	Disciplina	2023/2024	17-05-2024
		Curso		
		Licenciatura em Engenharia Informática Licenciatura em Segurança Informática em Redes de Computadores	1000	1000
		Unidade Curricular Programa de Engenharia		2750-06

Grupo II (6 valores)

Apresenta-se a definição de `Student`, que irá usar ao longo deste problema:

```
public class Student {
    private String name;
    private int id;

    public Student(String name, int id) {
        this.name = name;
        this.id = id;
    }

    // getters e setters
}
```

a) Complete os corpos dos construtores e a função `questionCount` na classe `Solution`. (1 valor)

```
public class Assignment {
    private String title;

    public Assignment(String title) {
        // TO DO
    }

    public class Solution extends Assignment {
        private int questionCount;

        public Solution(String title, int questionCount) {
            // TO DO
        }

        public int getQuestionCount() {
            // TO DO
        }
    }
}
```

b) Implemente o método `equals` para as classes `Assignment` e `Solution`. (2 valores)

c) Complete o construtor da classe `Grader`. (1 valor)

```
public class Grader {
    private String name;

    public Grader(String name) {
        // TO DO
    }
}
```

As definições apresentadas são válidas? Explique a sua resposta, considerando todas as particularidades de interfaces e classes abstratas em Java. Qual seria o resultado da execução do seguinte código?

```
public class TestAdvancedClass {
    public static void main(String[] args) {
        AdvancedClass ac = new AdvancedClass();
        ac.additionalMethod();
        ac.abstractMethod();
    }
}
```

e) Explique o conceito de encapsulamento em Programação Orientada a Objetos. Por que é importante e como pode ser implementado em Java? (2 valores) **✓**

f) Considere o seguinte código (2 valor)

```
class Animal {
    public void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    public void sound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    @Override
    public void sound() {
        System.out.println("Cat meows");
    }
}
```

```
public class TestPolymorphism {
    public static void main(String[] args) {
        Animal a1 = new Dog();
        Animal a2 = new Cat();
        a1.sound();
        a2.sound();
    }
}
```

Explique os conceitos de herança e polimorfismo com base no código acima. O que será impresso na consola quando o código for executado?

Prof. Ana Mendes

Assinatura